Deep Learning

Assignment 8

In this exercise sheet, we will take a closer look at backpropagation with a different activation function. After laying the theoretical foundation, we will implement backpropagation using native Python.

Question 1 Backpropagation (12 points)

A neural network with the following structure is given. *Note:* For this exercise we assume a batch size of 1, so that all vectors are *row vectors*.

$$\begin{aligned}
\boldsymbol{u}^{(1)} &= \boldsymbol{x}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)}\boldsymbol{x} \\
\boldsymbol{z} &= \tanh(\boldsymbol{u}^{(1)}), \\
\boldsymbol{u}^{(2)} &= \boldsymbol{z}^{(1)}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}, \\
\boldsymbol{y} &= \mathrm{Softmax}(\boldsymbol{u}^{(2)}), \\
\mathcal{L} &= H(\boldsymbol{y}', \boldsymbol{y}),
\end{aligned}$$

with $\tanh$ being the hyperbolic tangent function and H being the cross-entropy function.

In order to train the neural network, we need to adjust the trainable weights, i.e. the elements of the weight matrices and the bias vectors. For this purpose, we need to calculate the gradients:

$$\nabla_{\boldsymbol{W}^{(layer)}}\mathcal{L} \text{ and } \nabla_{\boldsymbol{b}^{(layer)}}\mathcal{L} \text{ for } layer = 1, 2$$

Revisit assignment 4: The derivative of the $\mathrm{Softmax}$ function has already been discussed, so you can use $\frac{\partial \mathcal{L}}{\partial u_i^{(2)}}$ without expanding it any further.

Remember: The chain rule is $\frac{\partial f}{\partial \boldsymbol{x}} = \sum_i \frac{\partial f}{\partial z_i}\frac{\partial z_i}{\partial \boldsymbol{x}}$ when the intermediate variable $\boldsymbol{z}$ is a vector. For this question you *don't* need to take into account that you are processing multiple inputs (a batch) at the same time. It might be helpful to solve this exercise by using the index method (show it for the i, j-th entry).

(a) Make a sketch of the neural network. (1 point)

(b) Use the chain rule to expand $\nabla_{\boldsymbol{W}^{(layer)}}\mathcal{L}$ and $\nabla_{\boldsymbol{b}^{(layer)}}\mathcal{L}$ for $layer = 1, 2$, into a chain of partial derivatives. (Do not calculate the derivatives yet.) (2 points)

(c) Calculate the derivatives. Show:

- $(\nabla_{\boldsymbol{W}^{(2)}}\mathcal{L})_{i,j} = z_i \dfrac{\partial \mathcal{L}}{\partial u_j^{(2)}}$
- $(\nabla_{\boldsymbol{b}^{(2)}}\mathcal{L})_i = \dfrac{\partial \mathcal{L}}{\partial u_i^{(2)}}$
- $(\nabla_{\boldsymbol{W}^{(1)}}\mathcal{L})_{i,j} = x_i(1 - z_j^2)\left(\dfrac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}}\boldsymbol{W}^{(2)^T}\right)_j$

- $(\nabla_{\boldsymbol{b}^{(1)}}\mathcal{L})_i = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(1)}}\right)_i = (1 - z_i^2)\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}}\boldsymbol{W}^{(2)^T}\right)_i$

(5 points)

(d) Suppose $\boldsymbol{y}$ has two features, and

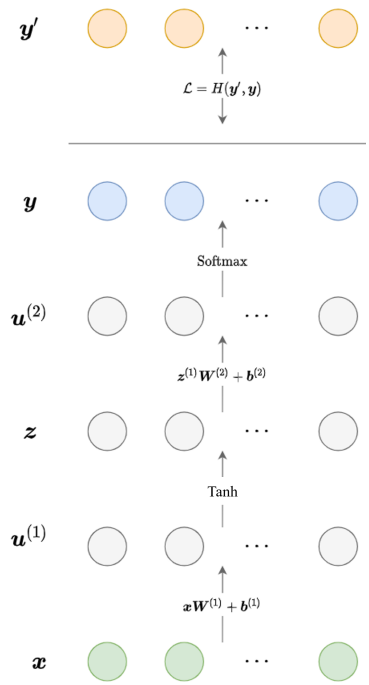
$$\boldsymbol{W}^{(1)} = \begin{pmatrix} 0.5 & 0.4 & 0.3 \\ 0.2 & 0.1 & 0 \end{pmatrix}$$
$$\boldsymbol{b}^{(1)} = (0, 0, 0)$$
$$\boldsymbol{W}^{(2)} = \begin{pmatrix} 0.5 & 0.4 \\ 0.3 & 0.2 \\ 0.1 & 0 \end{pmatrix}$$
$$\boldsymbol{b}^{(2)} = (0, 0)$$

First, calculate one forward pass with $\boldsymbol{x} = (1, 0)$. Then perform one update step for the ground truth label $\boldsymbol{y} = (0, 1)$ (backpropagation, learning rate $= 1$). Finally calculate another forward pass and compare the two results. (3 points)

(e) Find common subexpressions that can be used to avoid duplicate calculations. Write down the order of calculations to compute all the gradients efficiently. (1 point)

<u>Solution:</u>

(a)



(b)

$$\nabla_{\boldsymbol{W}^{(2)}}\mathcal{L} = \frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(2)}} \overset{\text{chain rule}}{=} \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \frac{\partial u_k^{(2)}}{\partial \boldsymbol{W}^{(2)}}$$

$$\nabla_{\boldsymbol{b}^{(2)}}\mathcal{L} = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \frac{\partial u_k^{(2)}}{\partial \boldsymbol{b}^{(2)}}$$

$$\begin{aligned}\nabla_{\boldsymbol{W}^{(1)}}\mathcal{L} &= \sum_k \sum_l \sum_m \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \frac{\partial u_k^{(2)}}{\partial z_l} \frac{\partial z_l}{\partial u_m^{(1)}} \frac{\partial u_m^{(1)}}{\partial \boldsymbol{W}^{(1)}} \\ &= \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \sum_l \frac{\partial u_k^{(2)}}{\partial z_l} \sum_m \frac{\partial z_l}{\partial u_m^{(1)}} \frac{\partial u_m^{(1)}}{\partial \boldsymbol{W}^{(1)}}\end{aligned}$$

$$\nabla_{\boldsymbol{b}^{(1)}}\mathcal{L} = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \sum_l \frac{\partial u_k^{(2)}}{\partial z_l} \sum_m \frac{\partial z_l}{\partial u_m^{(1)}} \frac{\partial u_m^{(1)}}{\partial \boldsymbol{b}^{(1)}}$$

(c)

First we repeat two rules:

$$\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}}\right)_i = \frac{\partial \mathcal{L}}{\partial b_i}$$

and

$$\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}}\right)_{i,j} = \frac{\partial \mathcal{L}}{\partial W_{i,j}}$$

1) $\nabla_{\boldsymbol{W}^{(2)}}\mathcal{L}$

$$\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(2)}}\right)_{i,j} = \left(\sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \frac{\partial u_k^{(2)}}{\partial \boldsymbol{W}^{(2)}}\right)_{i,j} = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \frac{\partial u_k^{(2)}}{\partial W_{ij}^{(2)}} \tag{1}$$

Here we know the first factor of the sum but still have to calculate

$$\frac{\partial u_k^{(2)}}{\partial W_{i,j}^{(2)}} = \frac{\partial \left(\boldsymbol{z}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)}\right)_k}{\partial W_{i,j}^{(2)}} = \frac{\partial \left(\boldsymbol{z}\boldsymbol{W}^{(2)}\right)_k}{\partial W_{i,j}^{(2)}} + \frac{\partial b_k^{(2)}}{\partial W_{i,j}^{(2)}} = \frac{\partial \left(\sum_l z_l \cdot W_{lk}\right)}{\partial W_{i,j}^{(2)}} = z_i \delta_{k,j}$$

because

$$\frac{\partial \left(\sum_l z_l \cdot W_{lk}\right)}{\partial W_{i,j}^{(2)}} = \begin{cases} z_i & \text{if } l = i \text{ and } k = j \\ 0 & \text{otherwise} \end{cases}$$

and so (1) is equal to

$$\sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} z_i \delta_{k,j} = \frac{\partial \mathcal{L}}{\partial u_j^{(2)}} z_i$$

In general we can write

$$\boxed{(\nabla_{\boldsymbol{W}^{(2)}} \mathcal{L})_{i,j} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(2)}} \right)_{i,j} = z_i \frac{\partial \mathcal{L}}{\partial u_j^{(2)}}}$$

2) $\nabla_{\boldsymbol{b}^{(2)}} \mathcal{L}$

$$\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(2)}} \right)_i = \left(\sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \frac{\partial u_k^{(2)}}{\partial \boldsymbol{b}^{(2)}} \right)_i = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \left(\frac{\partial \left(\boldsymbol{z} \boldsymbol{W}^{(2)} \right)_k}{\partial b_i^{(2)}} + \frac{\partial b_k^{(2)}}{\partial b_i^{(2)}} \right) = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \delta_{k,i} = \frac{\partial \mathcal{L}}{\partial u_i^{(2)}}$$

In general we can write

$$\boxed{(\nabla_{\boldsymbol{b}^{(2)}} \mathcal{L})_i = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(2)}} \right)_i = \frac{\partial \mathcal{L}}{\partial u_i^{(2)}}}$$

3) $\nabla_{\boldsymbol{W}^{(1)}} \mathcal{L}$

$$\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(1)}} \right)_{i,j} = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \sum_l \frac{\partial u_k^{(2)}}{\partial z_l^{(1)}} \sum_m \frac{\partial z_l^{(1)}}{\partial u_m^{(1)}} \frac{\partial u_m^{(1)}}{\partial W_{i,j}^{(1)}} \tag{2}$$

We calculate the individual parts

- $\dfrac{\partial u_k^{(2)}}{\partial z_l^{(2)}} = \dfrac{\partial \left(\sum_n z_n^{(1)} W_{n,k}^{(2)} \right)}{\partial z_l^{(1)}} = W_{l,k}^{(2)}$
- $\dfrac{\partial z_l^{(1)}}{\partial u_m^{(1)}} = \dfrac{\partial \tanh(u_l^{(1)})}{\partial u_m^{(1)}} = (1 - \tanh^2(u_l^{(1)}))\delta_{l,m} = (1 - z_l^2)\delta_{l,m}$
- $\dfrac{\partial u_m^{(1)}}{\partial W_{i,j}^{(1)}} = x_i \delta_{m,j}$

If we put this in formula (2), we get

$$\sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \sum_l W_{l,k}^{(2)} \sum_m (1 - z_l^2) \overbrace{\delta_{l,m} x_i \delta_{m,j}}^{\text{only } \neq 0 \text{ for } l=m=j} = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} W_{j,k}^{(2)} (1 - z_j^2) x_i$$

$$= x_i (1 - z_j^2) \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} W_{k,j}^{(2)^T}$$

$$= x_i (1 - z_j^2) \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}} \boldsymbol{W}^{(2)^T} \right)_j$$

In general we can write

$$\boxed{(\nabla_{\boldsymbol{W}^{(1)}} \mathcal{L})_{i,j} = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{W}^{(1)}} \right)_{i,j} = x_i (1 - z_j^2) \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}} \boldsymbol{W}^{(2)^T} \right)_j}$$

4) $\nabla_{\boldsymbol{b}^{(1)}} \mathcal{L}$

$$\left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(1)}} \right)_i = \sum_k \frac{\partial \mathcal{L}}{\partial u_k^{(2)}} \sum_l \frac{\partial u_k^{(2)}}{\partial z_l^{(1)}} \sum_m \frac{\partial z_l^{(1)}}{\partial u_m^{(1)}} \frac{\partial u_m^{(1)}}{\partial b_i^{(1)}} = (1 - z_j^2) \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}} \boldsymbol{W}^{(2)^T} \right)_j$$

- $\dfrac{\partial u_m^{(1)}}{\partial b_i^{(1)}} = \delta_{m,i}$ (identically to above except for the term x_i)

In general we can write

$$\boxed{(\nabla_{\boldsymbol{b}^{(1)}} \mathcal{L})_i = \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{b}^{(1)}} \right)_i = (1 - z_i^2) \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}} \boldsymbol{W}^{(2)^T} \right)_i}$$

(d) We set:

- $\texttt{dLdu2} := \frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}}$
- $\tanh(\boldsymbol{u}^{(1)}) = \boldsymbol{z}$
- $\texttt{dLdb1} := \left(\texttt{dLdu2} \cdot \boldsymbol{W}^{(2)^T} \right) * (1 - \boldsymbol{z}^2)$

(e) We calculate one forward pass:

$$\boldsymbol{u}^{(1)} = \boldsymbol{x}\boldsymbol{W}^{(1)} + \boldsymbol{b}^{(1)} = (1,0)\begin{pmatrix} 0.5 & 0.4 & 0.3 \\ 0.2 & 0.1 & 0 \end{pmatrix} + (0,0,0) = (0.5, 0.4, 0.3)$$

$$\boldsymbol{z} = \tanh(\boldsymbol{u}^{(1)}) = (\tanh(0.5), \tanh(0.4), \tanh(0.3)) \approx (0.4621, 0.3799, 0.2913)$$

$$\boldsymbol{u}^{(2)} = \boldsymbol{z}\boldsymbol{W}^{(2)} + \boldsymbol{b}^{(2)} = (0.4621, 0.3799, 0.2913)\begin{pmatrix} 0.5 & 0.4 \\ 0.3 & 0.2 \\ 0.1 & 0 \end{pmatrix} + (0,0) \approx (0.3742, 0.2608)$$

$$\boldsymbol{y} = \texttt{Softmax}(\boldsymbol{u}^{(2)}) = \left(\frac{e^{0.3742}}{e^{0.3742} + e^{0.2608}}, \frac{e^{0.2608}}{e^{0.3742} + e^{0.2608}} \right) \approx (0.5283, 0.4717)$$

Now we calculate the necessary derivatives for the backward pass.

First we calculate

$$\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}} \boldsymbol{W}^{(2)^T} = (0.5283, -0.5283) \cdot \begin{pmatrix} 0.5 & 0.3 & 0.1 \\ 0.4 & 0.2 & 0 \end{pmatrix} = (0.0528, 0.0528, 0.0528)$$

Then,

$$
\begin{aligned}
\nabla_{\boldsymbol{W}^{(2)}}\mathcal{L} &= \boldsymbol{z}^T \cdot \left(\texttt{Softmax}(u_0^{(2)}), \texttt{Softmax}(u_1^{(2)}) - 1\right) \\
&= \begin{pmatrix} 0.4621 \\ 0.3799 \\ 0.2913 \end{pmatrix} \cdot (\texttt{Softmax}(0.3742), \texttt{Softmax}(0.2608) - 1) \\
&= \begin{pmatrix} 0.4621 \\ 0.3799 \\ 0.2913 \end{pmatrix} \cdot (0.5283, -0.5283) \\
&= \begin{pmatrix} 0.2441 & -0.2441 \\ 0.2007 & -0.2007 \\ 0.1539 & -0.1539 \end{pmatrix}
\end{aligned}
$$

$$
\nabla_{\boldsymbol{b}^{(2)}}\mathcal{L} = (0.5283, -0.5283)
$$

$$
\begin{aligned}
\nabla_{\boldsymbol{W}^{(1)}}\mathcal{L} &= \boldsymbol{x}^T \left((1 - \boldsymbol{z}^2) * \left(\frac{\partial \mathcal{L}}{\partial \boldsymbol{u}^{(2)}} \boldsymbol{W}^{(2)^T}\right)\right) \\
&= \begin{pmatrix} 1 \\ 0 \end{pmatrix} \cdot \begin{pmatrix} (1 - 0.4621^2) \cdot 0.0528 \\ (1 - 0.3799^2) \cdot 0.0528 \\ (1 - 0.2913^2) \cdot 0.0528 \end{pmatrix}^T \\
&= \begin{pmatrix} 1 \\ 0 \end{pmatrix} (0.0415, 0.0452, 0.0483) \\
&= \begin{pmatrix} 0.0415 & 0.0452 & 0.0483 \\ 0 & 0 & 0 \end{pmatrix}
\end{aligned}
$$

$$
\nabla_{\boldsymbol{b}^{(1)}}\mathcal{L} = (0.0415, 0.0452, 0.0483)
$$

And now:

$$
\begin{aligned}
\boldsymbol{W}_{new}^{(2)} &= \boldsymbol{W}^{(2)} - \nabla_{\boldsymbol{W}^{(2)}}\mathcal{L} = \begin{pmatrix} 0.2559 & 0.6441 \\ 0.0993 & 0.4007 \\ -0.0539 & 0.1539 \end{pmatrix} \\
\boldsymbol{b}_{new}^{(2)} &= \boldsymbol{b}^{(2)} - \nabla_{\boldsymbol{b}^{(2)}}\mathcal{L} = (-0.5283, 0.5283) \\
\boldsymbol{W}_{new}^{(1)} &= \boldsymbol{W}^{(1)} - \nabla_{\boldsymbol{W}^{(1)}}\mathcal{L} = \begin{pmatrix} 0.4585 & 0.3548 & 0.2517 \\ 0.2 & 0.1 & 0. \end{pmatrix} \\
\boldsymbol{b}_{new}^{(1)} &= \boldsymbol{b}^{(1)} - \nabla_{\boldsymbol{b}^{(1)}}\mathcal{L} = (-0.0415, -0.0452, -0.0483)
\end{aligned}
$$

With this we calculate another forward pass:

$$\begin{aligned}\boldsymbol{u}_{new}^{(1)} &= \boldsymbol{x}\boldsymbol{W}_{new}^{(1)} + \boldsymbol{b}_{new}^{(1)} = (1,0)\begin{pmatrix}0.4585 & 0.3548 & 0.2517\\ 0.2 & 0.1 & 0.\end{pmatrix} + (-0.0415, -0.0452, -0.0483)\\ &= (0.4170, 0.3096, 0.2034)\end{aligned}$$

$$\boldsymbol{z}_{new} = \tanh(\boldsymbol{u}_{new}^{(1)}) = (\tanh(0.4170), \tanh(0.3096), \tanh(0.2034)) \approx (0.3943, 0.3001, 0.2007)$$

$$\begin{aligned}\boldsymbol{u}_{new}^{(2)} &= \boldsymbol{z}_{new}\boldsymbol{W}_{new}^{(2)} + \boldsymbol{b}_{new}^{(2)}\\ &= (0.3943, 0.3001, 0.2007)\begin{pmatrix}0.2559 & 0.6441\\ 0.0993 & 0.4007\\ -0.0539 & 0.1539\end{pmatrix} + (-0.5283, 0.5283)\\ &\approx (-0.4084, 0.9334)\end{aligned}$$

$$\boldsymbol{y}_{new} = \texttt{Softmax}(\boldsymbol{u}_{new}^{(2)}) = \left(\frac{e^{-0.4084}}{e^{-0.4084}+e^{0.9334}}, \frac{e^{0.9334}}{e^{-0.4084}+e^{0.9334}}\right) \approx (0.2070, 0.7930)$$

Question 2 Implementing Backpropagation 1/2 (4 points)

Complete the Jupyter notebook `backprop_1.ipynb`. You will implement what you have calculated by hand in Question 1 (d). Check whether your results match.

Additionally let the neural network train to classify samples in `AND` and `XOR`. The first neuron of $\boldsymbol{y}$ should represent `AND`, the second neuron `XOR`. For this, use the four samples (left) with their corresponding ground truth (right):

$$\begin{aligned}\boldsymbol{x}_0 &= (0,1) & \qquad \boldsymbol{y}_0 &= (0,1)\\ \boldsymbol{x}_1 &= (1,1) & \qquad \boldsymbol{y}_1 &= (1,0)\\ \boldsymbol{x}_2 &= (1,0) & \qquad \boldsymbol{y}_2 &= (0,1)\\ \boldsymbol{x}_3 &= (0,0) & \qquad \boldsymbol{y}_3 &= (1,0)\end{aligned}$$

Question 3 Implementing Backpropagation 2/2 (4 points)

Now we want leave the previous toy example behind us and use the same network for a well known tasks: classification on the MNIST dataset. The Jupyter notebook `backprop_2.ipynb` contains most of the code for training a 1-hidden-layer neural network on the MNIST dataset. Open the notebook and go through it. You will be instructed to add the appropriate operations to calculate the partial derivatives and the gradients that are then used in gradient descent. Make sure you avoid computing things twice by pre-computing the common subexpressions.

For this task you need to take into account that the batches consist of multiple images, so the input vector $\boldsymbol{x}$ is extended to a design matrix over a mini batch and similarly the other vectors. This means that you have to average over the batch at the end to compute the gradient.